

(4/6) AI in Multiple GPUs: Grad Accum & Data Parallelism

Lorenzo Cesconetto

2025-10-12

Table of contents

Part 4/6: Gradient Accumulation & Distributed Data Parallelism (DDP)	1
Introduction	1
A Quick Neural Network Refresher	1
Gradient Accumulation	2
Distributed Data Parallelism (DDP)	3
Implementing DDP from scratch in PyTorch	4
Conclusion	6

Part 4/6: Gradient Accumulation & Distributed Data Parallelism (DDP)

This article is part of a series about distributed AI across multiple GPUs:

- Part 1: Understanding the Host and Device Paradigm
- Part 2: Point-to-Point and Collective Operations
- Part 3: How GPUs Communicate
- **Part 4: Gradient Accumulation & Distributed Data Parallelism (DDP)** (this article)
- Part 5: ZeRO (*coming soon*)
- Part 6: Tensor Parallelism (*coming soon*)

Introduction

Distributed Data Parallelism (DDP) is the first parallelization method we'll look at. It's the baseline approach that's **always** used in distributed settings, and it's commonly combined with other parallelization techniques.

A Quick Neural Network Refresher

Training a neural network means running a forward pass, calculating the loss, backpropagating the gradients of each weight with respect to the loss function, and finally update weights (what we call an optimization step). In PyTorch, it typically looks like this:

```
import torch

def training_loop(
    model: torch.nn.Module,
    dataloader: torch.utils.data.DataLoader,
    optimizer: torch.optim.Optimizer,
):
    for i, batch in enumerate(dataloader):
        output = model(batch) # Forward pass
        output.loss.backward() # Backward pass (compute gradients)
        optimizer.step() # Update weights
        optimizer.zero_grad() # Clear gradients for the next step
```

Performing the optimization step on large amounts of data generally gives more accurate gradient estimates, leading to smoother training and potentially faster convergence. So ideally we would be taking each step after computing the gradients based on the entire training dataset. In practice, that's rarely feasible in Deep Learning scenarios, as it would take too long to compute. Instead, we work with small chunks called *mini-batches*.

- **Batch:** Refers to using the entire training set for one optimization step (i.e. weights update).
- **Mini-batch:** means using a small subset of the data for each optimization step.

This is where Gradient Accumulation and Data Parallelism comes into play. Although we don't use the entire dataset for each step, we can use these techniques to substantially increase our mini-batch size.

Gradient Accumulation

Here's how it works: if your mini-batch won't fit in memory, split it into *micro-batches*. For each micro-batch, run forward and backward passes, adding (accumulating) the computed gradients. Once all micro-batches are processed, perform a single optimization step using the summed gradients.

Notice Gradient Accumulation isn't a parallelization technique and doesn't require a multiple GPU setup.

Implementing Gradient Accumulation from scratch is straightforward. Here's what it looks like in a simple training loop:

```
import torch

def training_loop(
    model: torch.nn.Module,
    dataloader: torch.utils.data.DataLoader,
    optimizer: torch.optim.Optimizer,
    grad_accum_steps: int,
):
```

```

for i, batch in enumerate(dataloader):
    output = model(batch)
    output.loss.backward() # Gradients get accumulated (summed)

    # Only update weights after `grad_accum_steps` micro-batches
    if i % grad_accum_steps == 0:
        optimizer.step()
        optimizer.zero_grad()

```

Notice we're *sequentially* performing multiple forward and backward passes before each optimization step, which requires longer training times. It would be nice if we could speed this up by processing multiple micro-batches in *parallel*, well this is exactly what DDP does!

Distributed Data Parallelism (DDP)

For a fairly small number of GPUs (up to ~8) DDP scales almost linearly, which is optimal. That means that if you double the number of GPUs, you can almost halve the training time (we already discussed Linear-Scaling previously).

With DDP, multiple GPUs work together to process a larger effective batch size. The workflow looks like this:

1. Split the mini-batch across GPUs.
2. Each GPU runs its own forward and backward passes to compute gradients for its own data shard.
3. Use an **All-Reduce** operation (we previously learned about Collective ops) to average gradients across all GPUs.
4. Each GPU applies the same weight updates, keeping models in perfect sync.

This lets us train with much larger effective batch sizes, leading to more stable training and potentially faster convergence with little to none extra time.

Distributed Data Parallel (DDP)

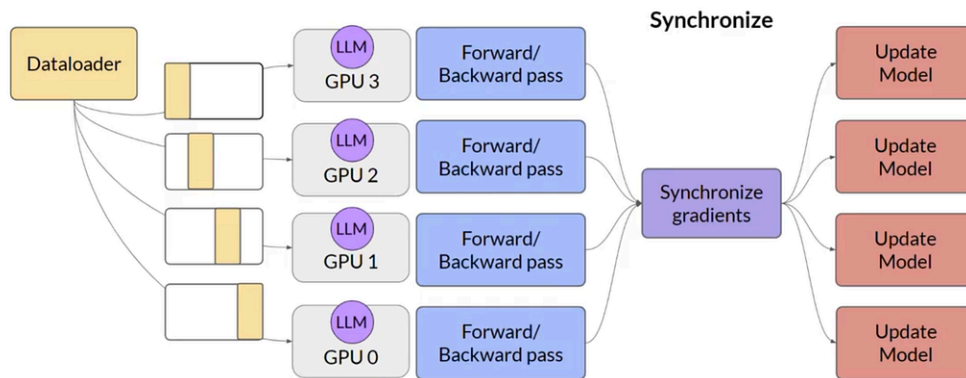


Figure 1: Source: Medium

Implementing DDP from scratch in PyTorch

Let's do this step-by-step. In this first iteration, we're only syncing the gradients.

```
import torch

class DDPMModelWrapper:
    def __init__(self, model: torch.nn.Module):
        self.model = model

    def __call__(self, *args, **kwargs):
        return self.model(*args, **kwargs)

    def sync_gradients(self):
        # Iterate over parameter matrices in the model
        for param in self.model.parameters():
            # Some parameters might be frozen and don't have gradients
            if param.grad is not None:
                # We sum and then divide since torch.distributed doesn't have
                # an average operation
                torch.distributed.all_reduce(param.grad.data,
                op=torch.distributed.ReduceOp.SUM)
                # Assuming each GPU received a equally sized mini-batch, we
                # can average
                # the gradients dividing by the number of GPUs (aka world
                # size)
                # By default the loss function already averages over the mini-
```

```
batch_size

param.grad.data /= torch.distributed.get_world_size()
```

Of course our model must be the same across all GPUs, otherwise we would be training different models! Let's improve our implementation by checking that all models are identical in `__init__`.

```
import torch

class DDPMModelWrapper:
    def __init__(self, model: torch.nn.Module):
        self.model = model
        for param in self.model.parameters():
            # We create a new tensor so it can receive the broadcast
            rank_0_param = param.data.clone()
            # Initially rank_0_param contains the values for the current rank
            torch.distributed.broadcast(rank_0_param, src=0)
            # After the broadcast rank_0_param variable is overwritten with
            the parameters from rank_0
            if not torch.equal(param.data, rank_0_param): # Now we compare
rank_x with rank_0
                raise ValueError("Model parameters are not the same across all
processes.")

    def __call__(self, *args, **kwargs):
        return self.model(*args, **kwargs)

    def sync_gradients(self):
        for param in self.model.parameters():
            if param.grad is not None:
                torch.distributed.all_reduce(param.grad.data,
op=torch.distributed.ReduceOp.SUM)
                param.grad.data /= torch.distributed.get_world_size()
```

A training loop using our DDPMModelWrapper and Gradient Accumulation would look like this:

```
def training_loop(
    ddp_model: DDPMModelWrapper,
    dataloader: torch.utils.data.DataLoader,
    optimizer: torch.optim.Optimizer,
    grad_accum_steps: int,
):
    for i, batch in enumerate(dataloader):
        output = ddp_model(batch)
        output.loss.backward()

        if i % grad_accum_steps == 0:
```

```
# Must sync gradients across GPUs *BEFORE* the optimization step
ddp_model.sync_gradients()
optimizer.step()
optimizer.zero_grad()
```

Conclusion

Congratulations for reading all the way to the end! In this post, you learned about:

- The importance of large batch sizes
- How Gradient Accumulation works and its limitations
- The DDP workflow and its benefits
- How to implement these techniques in PyTorch from scratch

In the next article, we'll explore ZeRO (Zero Redundancy Optimizer), a more advanced technique that builds upon DDP to further optimize VRAM memory usage.